Ejemplos sobre el uso de la instrucción `lea`

Arquitectura del Computador - LCC - FCEIA-UNR

29 de julio de 2026

Ejemplo 1

Dado el siguiente código en Assembler x86-64:

```
.data
a: .quad 0x1122334455667788

.text
.global main

main:
    leaq a, %rax      # Instrucción 1
    movq $a, %rax     # Instrucción 2
    ret
```

Compilar utilizando la opción `-g` para poder *debuggear* utilizando GDB:

```
$ gcc -g lea_ejemplo.s
```

Luego ejecutar utilizando GDB:

```
gdb ./a.out
```

Colocar un breakpoint en `main`:

```
(gdb) br main
Breakpoint 1 at 0x4004b6: file ejemplo_lea.s, line 9.
```

Ejecutar con el comando `run`:

```
(gdb) r
```

Ejecutar la primera línea con el comando `next`:

```
(gdb) n
```

Observar el contenido del registro `rax`:

```
(gdb) i r rax
rax            0x600880 6293632
```

Vemos que en el registro `rax` quedó almacenada la dirección de la etiqueta `a`, lo cual podemos verificar de manera adicional mediante el comando:

```
(gdb) info address a
Symbol "a" is at 0x600880 in a file compiled without debugging.
```

Ahora veamos que sucede cuando ejecutamos la siguiente línea:

```
(gdb) n
(gdb) i r rax
rax            0x600880 6293632
```

Vemos que la instrucción 2 es equivalente a la instrucción 1.

Como conclusión, hemos ejecutado dos línea de código equivalentes pero que utilizan instrucciones diferentes. En efecto, si bien las instrucciones 1 y 2 produjeron el mismo efecto, la instrucción `lea` es diferente a la dirección `mov` y permite realizar operaciones más complejas.

Ejemplo 2

Recordemos que la forma general de la instrucción `lea` es

```
leaq displacement(%base, %offset, multiplier), %dest
```

la cual corresponde a

```
%dest = displacement + %base + %offset * multiplier
```

donde `displacement` es una constante entera, `multiplier` es 2, 4 o 8 y `%dest`, `%offset` y `%base` son registros. Algunos de los operandos pueden no estar.

Ejemplos:

```
movq $100, %rax
movq $4, %rbx
leaq 16(%rax, %rbx, 2), %rcx        # rcx = 124
leaq 16(%rax, %rbx,), %rcx          # rcx = 120
leaq (%rax, %rbx,), %rcx            # rcx = 104
leaq (%rax), %rcx                   # rcx = 100
```

Además de brindar la posibilidad de realizar cálculos relativamente complejos[1], ese no es el propósito principal del la instrucción `lea`. El conjunto de instrucciones x86 fue diseñado para admitir lenguajes de alto nivel como Pascal y C, donde las matrices, especialmente las matrices de enteros, o estructuras pequeñas, son comunes.

Ejemplo 3

Consideremos, por ejemplo, una estructura que representa las coordenadas (x, y) de un punto:

```
#include <stdio.h>

struct Point
{
    int xcoord;
    int ycoord;
};

int main()
{
    struct Point p;
    p.xcoord = 10;
    p.ycoord = 20;
```

[1]Es importante señalar que la instrucción `lea` no modifica el registro de banderas a diferencia de las instrucciones aritméticas

```
    int y = p.ycoord;

    printf("y = %d, dirección de y = %p\n", y, &y);

    return 0;
}
```

La estructura queda así en memoria:

Campo	Offset	Tamaño
xcoord	0	4 bytes
ycoord	4	4 bytes

Tamaño total: 8 bytes.

Aquí vemos una implementación en ensamblador x86-64 (AT&T), versión simplificada tipo -O0 (sin optimizaciones):

```
.text
.global main
main:
    pushq %rbp
    movq %rsp, %rbp
    subq $16, %rsp          # reservar espacio

    # p.xcoord = 10
    movl $10, -8(%rbp)

    # p.ycoord = 20
    movl $20, -4(%rbp)

    # y = p.ycoord
    movl -4(%rbp), %eax
    movl %eax, -12(%rbp)

    # preparar argumentos para printf
    # printf("y = %d, dirección de y = %p\n", y, &y);

    movl -12(%rbp), %eax    # cargar y
    movl %eax, %esi         # 2do argumento: y

    leaq -12(%rbp), %rdx    # 3er argumento: &y

    leaq .LC0(%rip), %rdi   # 1er argumento: string

    movl $0, %eax           # requerido para funciones variádicas
    call printf

    movl $0, %eax
    leave
    ret
```

Ejemplo 4

Consideremos, ahora un arreglo de estructuras:

```
struct Point {
    int xcoord;
```

```
    int ycoord;
};

int main() {
    struct Point points[3] = {
        {1, 2},
        {3, 4},
        {5, 6}
    };

    int i = 1;
    int y = points[i].ycoord;
    int *p = &y;

    return 0;
}
```

Aquí vemos una implementación en ensamblador x86-64 (AT&T), versión simplificada tipo -O0 (sin optimizaciones):

```
.data
points:
.long 1, 2     # points[0]
.long 3, 4     # points[1]
.long 5, 6     # points[2]

.text
.global main
main:
    pushq %rbp
    movq %rsp, %rbp
    subq $16, %rsp           # espacio para i, y, p

    # i = 1
    movl $1, -4(%rbp)

    # cargar i en registro
    movl -4(%rbp), %ecx

    # rax = &points
    leaq points, %rax

    # y = points[i].ycoord
    # dirección efectiva: base + i*8 + 4
    movl 4(%rax,%rcx,8), %edx
    movl %edx, -8(%rbp)

    # p = &y
    leaq -8(%rbp), %rax
    movq %rax, -16(%rbp)

    movl $0, %eax
    leave
    ret
```